Ajaia Docs

A lightweight collaborative document editor: create and format documents, import files, and share them with other users as editors or viewers.

- **Live app:** https://ajaia-assessment-kappa.vercel.app (see `SUBMISSION.md`)
- **Stack:** Next.js 15 (App Router) + TypeScript, TipTap editor, Postgres via Drizzle ORM, Vitest. Deployed on Vercel with a free Neon Postgres database.

Demo accounts

Sign-in is mocked: choose an account on the sign-in screen, with no password. Use **Switch user** in the top bar to change accounts.

Account	Email	Seeded data
Alice Chen	alice@ajaia.test	Owns "Q4 planning" (shared with Bob as an editor)
Bob Martinez	bob@ajaia.test	Owns "Team handbook" (shared with Alice as a viewer)
Carol Singh	carol@ajaia.test	Nothing yet, so you can test sharing from scratch

You can also click **Create a demo account** on the sign-in screen to make your own (name and email, no password). The three accounts above always stay available. New accounts can share with them and receive shares like any other user.

A 2-minute review path

1. Sign in as Alice. "My documents" and "Shared with me" are listed separately, and each document has an Owner, Can edit or Can view badge.
2. Click **New document**, rename it in the title field, and type with formatting (headings, bold, italic, underline, lists). Reload the page: the content and formatting persist.
3. Click **Share**, and add `carol@ajaia.test` as **Can view**.
4. Click **Switch user**, then choose Carol. The document is under "Shared with me", and opening it shows a read-only view with no toolbar.
5. Back as Alice, click **Import file**, then upload a `.md`, `.txt` or `.docx` file. It opens as a new editable document. Sample files are in `samples/`.
6. **Conflict handling:** open "Q4 planning" as Alice in one browser and as Bob in another (or a private window). Type in Bob's window and wait for "Saved", then type in Alice's. Alice is told Bob saved first, and after **Load latest version** her own edits are kept beside the document.
7. Click **Download → Markdown** to export.

Features

Area	What works
Editing	Create, rename, edit, autosave (about 0.8 s after typing stops), with save status in the header. Ctrl/Cmd+S saves immediately. Formatting: H1–H3, bold, italic, underline, bulleted and numbered lists, quotes, left/center/right alignment, links (Ctrl/Cmd+K), clear formatting, undo and redo. Live word and character count.
File upload	Import a **.txt, .md or .docx file (max 5 MB)** as a new document. Formatting is kept where the editor supports it, and unsupported or unsafe markup (scripts, images, tables) is dropped. Other file types are rejected with a clear message.
Sharing	Each document has one owner. The owner shares by email as **editor** or **viewer**, can change a role, and can remove access. Editors can edit and rename. Only the owner can share or delete.
Persistence	Postgres. Content is stored as TipTap JSON, so formatting round-trips exactly.
Concurrent saves	Each save carries the version it was based on. If someone else saved first, the server rejects the save (HTTP 409) and names who saved. The editor then loads the latest version in place and keeps your unsaved edits alongside it, so nothing is lost. You can copy from them, **Use my version**, or discard them. Document lists show who edited last.
Export	**Download → Markdown (.md)**, generated on the server; anyone with read access can export. **Download → PDF** uses the browser's print-to-PDF with a print stylesheet that hides the app chrome.
Safety	Links are limited to http, https and mailto, checked in the editor, on import and on every save. A `javascript:` link sent directly to the API is rejected.

Run locally

Requirements: Node 20+ and a Postgres database (local, Docker, or a free Neon project).

```
npm install
cp .env.example .env.local      # then set DATABASE_URL and SESSION_SECRET
npm run db:setup                 # applies migrations and seeds the demo accounts (safe
to re-run)
npm run dev                      # http://localhost:3000
```

Postgres with Docker, if you don't have one running:

```
docker run -d --name ajaia-pg -e POSTGRES_PASSWORD=postgres -p 5432:5432 postgres:16
# DATABASE_URL=postgres://postgres:postgres@localhost:5432/postgres
```

Tests

```
npm test                                                  # unit tests (no database
needed)
TEST_DATABASE_URL=postgres://... npm test                 # also runs the Postgres
integration tests
```

- `tests/access.test.ts`: the full permission matrix (owner, editor, viewer, no access) for read, edit, rename, share and delete.
- `tests/import.test.ts`: .md, .txt and .docx conversion; unsafe markup is stripped; bad, empty and oversized files are rejected; content validation.
- `tests/markdown.test.ts`: Markdown export (marks, lists, nesting, escaping) and a round trip (import, then export, then import gives an identical document); link safety.
- `tests/users.integration.test.ts`: demo-account creation (normalized names and emails, duplicate emails rejected regardless of case, demo accounts always listed first, and sharing to and from a new account).
- `tests/sharing.integration.test.ts`: the real data layer against Postgres. Covers shared-with-me listings, role enforcement, role upgrades, stale-save conflicts (including who saved first), and revoking access. **Note:** it truncates the tables in the database you point it at, so use a separate test database.

Deploy (Vercel + Neon, both free)

1. Create a Neon project and copy the **pooled** connection string (it ends with `?sslmode=require`).
2. Import the repo in Vercel. Set the environment variables `DATABASE_URL` (the Neon string) and `SESSION_SECRET` (any random string of 32+ characters).
3. Deploy. Vercel runs the `vercel-build` script, which applies migrations, seeds the demo accounts, and then builds the app.

Project layout

```
src/app/            pages (sign-in, dashboard, editor) and API routes under src/app/api
src/components/     DocEditor (autosave and conflicts), Toolbar, ShareDialog, Dashboard
src/lib/access.ts   permission rules, used by every API route
src/lib/documents.ts data access; every function checks permissions first
src/lib/import.ts   file to document conversion
src/lib/markdown.ts document to Markdown export
src/lib/content.ts  document validation against the editor schema
src/db/             Drizzle schema and client; migrations are in drizzle/
scripts/db-setup.ts migrations and idempotent seed
```

See `ARCHITECTURE.md` for decisions and tradeoffs, and `AI_WORKFLOW.md` for how AI was used.